Mondragon Unibertsitatea

Faculty of Engineering

PROGRAMACIÓN APLICADA A LA IA

Trabajo Grupal
Pipeline de Análisis de Datos con Python

De un CSV en bruto a un informe visual: aplicando Numpy, Pandas, Matplotlib, Funciones, POO y Excepciones.

Ibon Soto Alsua
isoto@mondragon.edu

Índice

Mondragon Unibertsitatea
Faculty of Engineering

1. Objetivo del trabajo
2. Organización y grupos
3. Tu dataset
4. El flujo de datos esperado
5. Técnicas obligatorias
6. Ejemplo ilustrativo: punto de entrada
7. Entregables
8. Reglas y restricciones
9. Evaluación
10. Calendario y normas

1. Objetivo del trabajo

Mondragon Unibertsitatea
Faculty of Engineering

Diseñar y construir, en grupo, un pequeño pipeline de análisis de datos orientado a objetos que:

- cargue un CSV y valide que tiene un formato razonable,
- limpie los datos (nulos, duplicados, tipos, texto),
- los analice (estadísticas, filtros, agrupaciones),
- y genere una visualización clara del resultado.

La arquitectura (qué clases usáis, cómo se llaman, cómo se relacionan) la diseñáis VOSOTROS. Aquí solo se fija la idea general y las técnicas que tenéis que demostrar.

2. Organización y grupos

Mondragon Unibertsitatea
Faculty of Engineering

10 grupos de 3 personas.

Poneos de acuerdo en la arquitectura ANTES de repartir el trabajo: es más fácil integrar piezas si desde el principio sabéis qué recibe y qué devuelve cada clase.

3. Tu dataset

Mondragon Unibertsitatea
Faculty of Engineering

Cada grupo tiene asignado uno de estos 10 datasets reales:

- **Grupo 1:** `grupo01_marvel_comics.csv` — personajes de Marvel Comics (16.376 filas)
- **Grupo 2:** `grupo02_cine_test_bechdel.csv` — cine y el test de Bechdel (1.794 filas)
- **Grupo 3:** `grupo03_peliculas_biograficas.csv` — películas biográficas (761 filas)
- **Grupo 4:** `grupo04_carreras_universitarias.csv` — carreras universitarias y salarios (173 filas)
- **Grupo 5:** `grupo05_ranking_caramelos.csv` — ranking de caramelos de Halloween (85 filas)
- **Grupo 6:** `grupo06_futbol_audiencia_fifa.csv` — audiencia y economía del fútbol (191 filas)
- **Grupo 7:** `grupo07_clima_phoenix.csv` — clima de Phoenix, un año completo (365 filas)
- **Grupo 8:** `grupo08_bibliotecarios_empleo.csv` — empleo de bibliotecarios en EE.UU. (373 filas)
- **Grupo 9:** `grupo09_bob_ross_elementos.csv` — elementos pintados por Bob Ross (403 filas)
- **Grupo 10:** `grupo10_madmen_actores.csv` — actores de series de TV premiadas (248 filas)

4. El flujo de datos esperado

Mondragon Unibertsitatea
Faculty of Engineering



- Este es el flujo de datos que se espera: podéis resolverlo con 2, 3, 4 o más clases, y llamarlas como queráis.

- Lo que sí es evaluable es que las responsabilidades estén claramente separadas: una parte del código no debería mezclar “leer” con “limpiar” con “dibujar”.

- Además del flujo de datos, el diseño debe incluir excepciones propias para los errores esperables (dataset inválido, columna inexistente…) y funciones auxiliares reutilizables fuera de las clases.

5. Técnicas obligatorias

Mondragon Unibertsitatea
Faculty of Engineering

Con independencia de cómo diseñéis vuestras clases, en algún punto del código tiene que aparecer, de forma justificada (no forzada):

- Al menos una relación de herencia real, con uso correcto de super().
- Encapsulación: atributos protegidos/privados junto con @property o getters/setters.
- Al menos una función con *args o **kwargs, y otra con un parámetro con valor por defecto.
- Al menos 2 excepciones personalizadas (heredan de Exception) usadas, no solo declaradas.
- try / except / else / finally alrededor del flujo principal del programa.
- Mínimo 3 gráficos distintos, con título y ejes, guardados en una carpeta de resultados.

Cómo organicéis estas técnicas en clases y ficheros es decisión vuestra.

6. Ejemplo ilustrativo: punto de entrada

Mondragon Unibertsitatea
Faculty of Engineering

```
try:
    datos = CargadorDeDatos(RUTA).cargar()
    mi_limpiador = Limpiador()
    datos = mi_limpiador.limpiar(datos)
    # ... análisis y gráficos con vuestras propias clases ...
except (FileNotFoundError, DatasetInvalidoError) as e:
    print(f"No se han podido cargar los datos: {e}")
except ColumnaInexistenteError as e:
    print(f"Error en el análisis: {e}")
else:
    print("Pipeline ejecutado correctamente.")
finally:
    print("Fin de la ejecución.")
```

- CargadorDeDatos, mi_limpiador, DatasetInvalidoError… son nombres inventados para este ejemplo: es solo una ilustración del patrón try/except/else/finally, NO una plantilla ni unos nombres obligatorios.

- Vuestras clases, atributos y métodos pueden (y deben) llamarse de forma distinta.

7. Entregables

Mondragon Unibertsitatea
Faculty of Engineering

Un único .zip (o enlace a repositorio) por grupo, con:

- Módulos .py, organizados por responsabilidad (carga, limpieza, análisis, visualización, excepciones…),
- Carpeta resultados/ con los gráficos generados y el dataset limpio exportado a CSV,
- PPT que muestre cómo se ha desarrollado la solución con los resultados y conclusiones.

No se pide memoria ni informe extenso: el código, los resultados y el PPT bastan.

- El código debe ejecutarse de principio a fin sin errores no controlados.

8. Reglas y restricciones

Mondragon Unibertsitatea
Faculty of Engineering

- Solo librerías vistas en clase: pandas, numpy, matplotlib y librería estándar de Python.

- No se permite resolver el trabajo en un único fichero ni en un notebook: la separación en clases/ficheros es parte de la evaluación.

- **Cada grupo trabaja con un dataset distinto: no debería haber dos entregas que se parezcan, ni en resultados ni en arquitectura de clases.**

- Se valorará negativamente una arquitectura calcada de la de otro grupo, aunque el dataset sea diferente.

Mondragon Unibertsitatea

Faculty of Engineering

Eskerrik asko
Muchas gracias
Thank you